커스텀 프로토콜 등록 예제

개요

이 문서는 Rust libp2p에서 사용자 정의 프로토콜을 구현하고 등록하는 방법을 상세한 예제와 함께 설명합니다. multistream-select를 통한 프로토콜 네고시에이션과 NetworkBehaviour 트레이트를 활용한 프로토콜 통합 방법을 다룹니다.

1. 기본 커스텀 프로토콜 구현

1.1 간단한 Echo 프로토콜

```
use libp2p::{
core::{upgrade, InboundUpgrade, OutboundUpgrade, UpgradeInfo},
futures::{AsyncRead, AsyncReadExt, AsyncWrite, AsyncWriteExt},
swarm::{
ConnectionHandler, ConnectionHandlerEvent, ConnectionHandlerUpgrErr,
KeepAlive, NegotiatedSubstream, SubstreamProtocol,
},
StreamProtocol,
};
use std::{
collections::VecDeque,
io,
```

```
task::{Context, Poll},
time::Duration,
};

// 프로토콜 식별자 정의
const ECHO_PROTOCOL: StreamProtocol = StreamProtocol::new("/echo/1.0.0");

// Echo 프로토콜 구조체
#[derive(Debug, Clone)]
pub  struct EchoProtocol;

impl UpgradeInfo for EchoProtocol {
type Info = StreamProtocol;
type InfoIter = std::iter::Once<Self::Info>;

fn protocol_info(&self) -> Self::InfoIter {
std::iter::once(ECHO_PROTOCOL)
}
}

// 인바운드 업그레이드 구현
impl<TSocket> InboundUpgrade<TSocket> for EchoProtocol
where
```

```rust
TSocket: AsyncRead + AsyncWrite + Unpin + Send + 'static,
{
type Output = TSocket;
type Error = io::Error;
type Future = futures::future::Ready<Result<Self::Output, Self::Error>>;

fn upgrade_inbound(self, socket: TSocket, _: Self::Info) -> Self::Future {
futures::future::ready(Ok(socket))
}
}

// 아웃바운드 업그레이드 구현
impl<TSocket> OutboundUpgrade<TSocket> for EchoProtocol
where
TSocket: AsyncRead + AsyncWrite + Unpin + Send + 'static,
{
type Output = TSocket;
type Error = io::Error;
type Future = futures::future::Ready<Result<Self::Output, Self::Error>>;

fn upgrade_outbound(self, socket: TSocket, _: Self::Info) -> Self::Future {
futures::future::ready(Ok(socket))
}
```

```rust
}

// Echo 메시지 타입
#[derive(Debug, Clone)]
pub struct EchoMessage {
pub data: Vec<u8>,
}

// Echo 이벤트 타입
#[derive(Debug)]
pub enum EchoEvent {
MessageReceived { message: EchoMessage },
MessageSent { message: EchoMessage },
Error { error: io::Error },
}

// Echo ConnectionHandler 구현
pub struct EchoHandler {
inbound_substreams: VecDeque<NegotiatedSubstream>,
outbound_substreams: VecDeque<NegotiatedSubstream>,
pending_events: VecDeque<EchoEvent>,
keep_alive: KeepAlive,
}
```

```rust
impl EchoHandler {
pub fn new() -> Self {
Self {
inbound_substreams: VecDeque::new(),
outbound_substreams: VecDeque::new(),
pending_events: VecDeque::new(),
keep_alive: KeepAlive::Yes,
}
}

pub fn send_message(&mut self, message: EchoMessage) {
// 메시지 전송 로직은 poll에서 처리
self.pending_events.push_back(EchoEvent::MessageSent { message });
}
}

impl ConnectionHandler for EchoHandler {
type FromBehaviour = EchoMessage;
type ToBehaviour = EchoEvent;
type InboundProtocol = EchoProtocol;
type OutboundProtocol = EchoProtocol;
type InboundOpenInfo = ();
```

```
type OutboundOpenInfo = EchoMessage;


fn listen_protocol(&self) -> SubstreamProtocol<Self::InboundProtocol, Self::Inbo

SubstreamProtocol::new(EchoProtocol, ())

}


fn on_behaviour_event(&mut  self, event: Self::FromBehaviour) {

// Behaviour로부터 메시지를 받아 전송 준비

self.send_message(event);

}


fn connection_keep_alive(&self) -> bool {

matches!(self.keep_alive, KeepAlive::Yes)

}


fn poll(

&mut  self,

cx: &mut Context<'_>,

) -> Poll<

ConnectionHandlerEvent<

Self::OutboundProtocol,

Self::OutboundOpenInfo,

Self::ToBehaviour,
```

```
>,
> {
// 대기 중인 이벤트 처리
if  let Some(event) = self.pending_events.pop_front() {
return Poll::Ready(ConnectionHandlerEvent::NotifyBehaviour(event));
}


// 인바운드 서브스트림 처리
while  let Some(mut substream) = self.inbound_substreams.pop_front() {
let  mut buffer = vec![0u8; 1024];
match substream.poll_read(cx, &mut buffer) {
Poll::Ready(Ok(n)) if n > 0 => {
buffer.truncate(n);
let message = EchoMessage { data: buffer.clone() };
// Echo back the message
match substream.poll_write(cx, &buffer) {
Poll::Ready(Ok(_)) => {
return Poll::Ready(ConnectionHandlerEvent::NotifyBehaviour(
EchoEvent::MessageReceived { message }
));
}
Poll::Ready(Err(e)) => {
return Poll::Ready(ConnectionHandlerEvent::NotifyBehaviour(
```

```
EchoEvent::Error { error: e }
));
}
Poll::Pending => {
self.inbound_substreams.push_back(substream);
break;
}
}
}
Poll::Ready(Ok(_)) => {
// Connection closed
continue;
}
Poll::Ready(Err(e)) => {
return Poll::Ready(ConnectionHandlerEvent::NotifyBehaviour(
EchoEvent::Error { error: e }
));
}
Poll::Pending => {
self.inbound_substreams.push_back(substream);
break;
}
}
```

```
}

Poll::Pending
}

fn on_connection_event(
&mut  self,
event: libp2p::swarm::handler::ConnectionEvent<
Self::InboundProtocol,
Self::OutboundProtocol,
Self::InboundOpenInfo,
Self::OutboundOpenInfo,
>,
) {
match event {
libp2p::swarm::handler::ConnectionEvent::FullyNegotiatedInbound(
libp2p::swarm::handler::FullyNegotiatedInbound { protocol, .. }
) => {
self.inbound_substreams.push_back(protocol);
}
libp2p::swarm::handler::ConnectionEvent::FullyNegotiatedOutbound(
libp2p::swarm::handler::FullyNegotiatedOutbound { protocol, .. }
) => {
```

```
self.outbound_substreams.push_back(protocol);
}
_ => {}
}
}
}
```

1.2 Echo NetworkBehaviour 구현

```
use libp2p::{
swarm::{
behaviour::ConnectionEstablished, behaviour::ConnectionClosed,
NetworkBehaviour, ToSwarmEvent, ConnectionId, THandlerInEvent, THandlerOutEvent,
},
PeerId, Multiaddr,
};
use std::collections::{HashMap, VecDeque};

// Echo Behaviour 이벤트
#[derive(Debug)]
pub  enum EchoBehaviourEvent {
MessageReceived {
```

```
peer: PeerId,
message: EchoMessage,
},
MessageSent {
peer: PeerId,
message: EchoMessage,
},
Error {
peer: PeerId,
error: io::Error,
},
}

// Echo NetworkBehaviour 구현
pub struct EchoBehaviour {
events: VecDeque<EchoBehaviourEvent>,
connected_peers: HashMap<PeerId, ConnectionId>,
}

impl EchoBehaviour {
pub fn new() -> Self {
Self {
events: VecDeque::new(),
```

```
connected_peers: HashMap::new(),
}
}

pub fn send_message(&mut self, peer: PeerId, message: EchoMessage) -> Result<(
if self.connected_peers.contains_key(&peer) {
// 메시지 전송은 poll에서 처리
Ok(())
} else {
Err("Peer not connected")
}
}
}

impl NetworkBehaviour for EchoBehaviour {
type ConnectionHandler = EchoHandler;
type ToSwarm = EchoBehaviourEvent;

fn handle_established_inbound_connection(
&mut self,
_connection_id: ConnectionId,
_peer: PeerId,
_local_addr: &Multiaddr,
```

```
_remote_addr: &Multiaddr,

) -> Result<libp2p::swarm::THandler<Self>, libp2p::swarm::ConnectionDenied> {

Ok(EchoHandler::new())

}



fn handle_established_outbound_connection(

&mut  self,

_connection_id: ConnectionId,

_peer: PeerId,

_addr: &Multiaddr,

_role_override: libp2p::core::Endpoint,

) -> Result<libp2p::swarm::THandler<Self>, libp2p::swarm::ConnectionDenied> {

Ok(EchoHandler::new())

}



fn on_swarm_event(&mut  self, event: libp2p::swarm::FromSwarm<Self::ConnectionHa

match event {

libp2p::swarm::FromSwarm::ConnectionEstablished(ConnectionEstablished {

peer_id,

connection_id,

..

}) => {

self.connected_peers.insert(peer_id, connection_id);
```

```
}
libp2p::swarm::FromSwarm::ConnectionClosed(ConnectionClosed {
peer_id,
connection_id,
..
}) => {
if  let Some(conn_id) = self.connected_peers.get(&peer_id) {
if *conn_id == connection_id {
self.connected_peers.remove(&peer_id);
}
}
}
_ => {}
}
}


fn on_connection_handler_event(
&mut  self,
peer_id: PeerId,
_connection_id: ConnectionId,
event: THandlerOutEvent<Self>,
) {
match event {
```

```
EchoEvent::MessageReceived { message } => {
self.events.push_back(EchoBehaviourEvent::MessageReceived {
peer: peer_id,
message,
});
}
EchoEvent::MessageSent { message } => {
self.events.push_back(EchoBehaviourEvent::MessageSent {
peer: peer_id,
message,
});
}
EchoEvent::Error { error } => {
self.events.push_back(EchoBehaviourEvent::Error {
peer: peer_id,
error,
});
}
}
}

fn poll(
&mut  self,
```

```
_cx: &mut Context<'_>,

) -> Poll<ToSwarmEvent<Self::ToSwarm, THandlerInEvent<Self>>> {

if  let Some(event) = self.events.pop_front() {

return Poll::Ready(ToSwarmEvent::GenerateEvent(event));

}

Poll::Pending

}

}
```

2. 고급 커스텀 프로토콜 예제

2.1 Request-Response 패턴 프로토콜

```
use libp2p::{

core::{upgrade, InboundUpgrade, OutboundUpgrade, UpgradeInfo},

futures::{AsyncRead, AsyncReadExt, AsyncWrite, AsyncWriteExt},

StreamProtocol,

};

use serde::{Deserialize, Serialize};

use std::io;


// Request-Response 프로토콜 식별자

const REQUEST_RESPONSE_PROTOCOL: StreamProtocol = StreamProtocol::new("/request-
```

```rust
// Request 및 Response 타입 정의
#[derive(Debug, Clone, Serialize, Deserialize)]
pub struct CustomRequest {
pub id: u64,
pub data: String,
pub timestamp: u64,
}

#[derive(Debug, Clone, Serialize, Deserialize)]
pub struct CustomResponse {
pub request_id: u64,
pub result: String,
pub success: bool,
}

// Request-Response 프로토콜 구조체
#[derive(Debug, Clone)]
pub struct RequestResponseProtocol;

impl UpgradeInfo for RequestResponseProtocol {
type Info = StreamProtocol;
type InfoIter = std::iter::Once<Self::Info>;
```

```
fn protocol_info(&self) -> Self::InfoIter {
std::iter::once(REQUEST_RESPONSE_PROTOCOL)
}
}

// 인바운드 업그레이드 (서버 역할)
impl<TSocket> InboundUpgrade<TSocket> for RequestResponseProtocol
where
TSocket: AsyncRead + AsyncWrite + Unpin + Send + 'static,
{
type Output = RequestResponseInbound<TSocket>;
type Error = io::Error;
type Future = futures::future::Ready<Result<Self::Output, Self::Error>>;

fn upgrade_inbound(self, socket: TSocket, _: Self::Info) -> Self::Future {
futures::future::ready(Ok(RequestResponseInbound::new(socket)))
}
}

// 아웃바운드 업그레이드 (클라이언트 역할)
impl<TSocket> OutboundUpgrade<TSocket> for RequestResponseProtocol
where
```

```
TSocket: AsyncRead + AsyncWrite + Unpin + Send + 'static,
{
type Output = RequestResponseOutbound<TSocket>;
type Error = io::Error;
type Future = futures::future::Ready<Result<Self::Output, Self::Error>>;

fn upgrade_outbound(self, socket: TSocket, _: Self::Info) -> Self::Future {
futures::future::ready(Ok(RequestResponseOutbound::new(socket)))
}
}

// 인바운드 스트림 래퍼
pub struct RequestResponseInbound<TSocket> {
socket: TSocket,
}

impl<TSocket> RequestResponseInbound<TSocket>
where
TSocket: AsyncRead + AsyncWrite + Unpin,
{
pub fn new(socket: TSocket) -> Self {
Self { socket }
}
```

```
pub  async  fn receive_request(&mut  self) -> Result<CustomRequest, io::Error> {

let  mut length_bytes = [0u8; 4];

self.socket.read_exact(&mut length_bytes).await?;

let length = u32::from_be_bytes(length_bytes) as usize;



let  mut buffer = vec![0u8; length];

self.socket.read_exact(&mut buffer).await?;



bincode::deserialize(&buffer)

.map_err(|e| io::Error::new(io::ErrorKind::InvalidData, e))

}



pub  async  fn send_response(&mut  self, response: CustomResponse) -> Result<(),

let data = bincode::serialize(&response)

.map_err(|e| io::Error::new(io::ErrorKind::InvalidData, e))?;

let length = data.len() as u32;

self.socket.write_all(&length.to_be_bytes()).await?;

self.socket.write_all(&data).await?;

self.socket.flush().await?;

Ok(())

}

}
```

```rust
// 아웃바운드 스트림 래퍼
pub struct RequestResponseOutbound<TSocket> {
socket: TSocket,
}

impl<TSocket> RequestResponseOutbound<TSocket>
where
TSocket: AsyncRead + AsyncWrite + Unpin,
{
pub fn new(socket: TSocket) -> Self {
Self { socket }
}

pub async fn send_request(&mut self, request: CustomRequest) -> Result<(), io
let data = bincode::serialize(&request)
.map_err(|e| io::Error::new(io::ErrorKind::InvalidData, e))?;
let length = data.len() as u32;
self.socket.write_all(&length.to_be_bytes()).await?;
self.socket.write_all(&data).await?;
self.socket.flush().await?;
Ok(())
}
```

```
pub  async  fn receive_response(&mut  self) -> Result<CustomResponse, io::Error>

let  mut length_bytes = [0u8; 4];

self.socket.read_exact(&mut length_bytes).await?;

let length = u32::from_be_bytes(length_bytes) as usize;



let  mut buffer = vec![0u8; length];

self.socket.read_exact(&mut buffer).await?;



bincode::deserialize(&buffer)

.map_err(|e| io::Error::new(io::ErrorKind::InvalidData, e))

}

}
```

3. 프로토콜 통합 및 사용 예제

3.1 Swarm에 커스텀 프로토콜 통합

```
use libp2p::{

identity, noise, tcp, yamux, Multiaddr, PeerId, Swarm, SwarmBuilder,

};

use std::error::Error;
```

```rust
// 통합된 Behaviour 구조체
#[derive(NetworkBehaviour)]
pub struct MyBehaviour {
echo: EchoBehaviour,
identify: libp2p::identify::Behaviour,
kademlia: libp2p::kad::Kademlia<libp2p::kad::store::MemoryStore>,
}

impl MyBehaviour {
pub fn new(local_key: &identity::Keypair) -> Self {
let local_peer_id = PeerId::from(local_key.public());
// Identify 설정
let identify = libp2p::identify::Behaviour::new(
libp2p::identify::Config::new("/my-app/1.0.0".to_string(), local_key.public())
);

// Kademlia 설정
let store = libp2p::kad::store::MemoryStore::new(local_peer_id);
let kademlia = libp2p::kad::Kademlia::new(local_peer_id, store);

Self {
echo: EchoBehaviour::new(),
identify,
```

```
kademlia,
}
}
}

// Swarm 생성 및 설정
pub  async  fn create_swarm() -> Result<Swarm<MyBehaviour>, Box<dyn Error>> {
let local_key = identity::Keypair::generate_ed25519();
let local_peer_id = PeerId::from(local_key.public());

let swarm = SwarmBuilder::with_existing_identity(local_key)
.with_tokio()
.with_tcp(
tcp::Config::default(),
noise::Config::new,
yamux::Config::default,
)?
.with_behaviour(|key| MyBehaviour::new(key))?
.with_swarm_config(|c| c.with_idle_connection_timeout(Duration::from_secs(60)))
.build();

Ok(swarm)
}
```

```rust
// 사용 예제
pub  async  fn run_example() -> Result<(), Box<dyn Error>> {
let  mut swarm = create_swarm().await?;


// 리스닝 주소 설정
swarm.listen_on("/ip4/0.0.0.0/tcp/0".parse()?)?;


// 이벤트 루프
loop {
match swarm.select_next_some().await {
SwarmEvent::NewListenAddr { address, .. } => {
println!("Listening on {}", address);
}
SwarmEvent::Behaviour(MyBehaviourEvent::Echo(echo_event)) => {
match echo_event {
EchoBehaviourEvent::MessageReceived { peer, message } => {
println!("Received echo from {}: {:?}", peer, message);
}
EchoBehaviourEvent::MessageSent { peer, message } => {
println!("Sent echo to {}: {:?}", peer, message);
}
EchoBehaviourEvent::Error { peer, error } => {
```

```
println!("Echo error with {}: {}", peer, error);
}
}
}
SwarmEvent::Behaviour(MyBehaviourEvent::Identify(identify_event)) => {
println!("Identify event: {:?}", identify_event);
}
SwarmEvent::Behaviour(MyBehaviourEvent::Kademlia(kad_event)) => {
println!("Kademlia event: {:?}", kad_event);
}
SwarmEvent::ConnectionEstablished { peer_id, .. } => {
println!("Connected to {}", peer_id);
// Echo 메시지 전송 예제
let message = EchoMessage {
data: b"Hello from custom protocol!".to_vec(),
};
if  let Err(e) = swarm.behaviour_mut().echo.send_message(peer_id, message) {
println!("Failed to send echo message: {}", e);
}
}
_ => {}
}
}
```

```
}
```

3.2 프로토콜 버전 관리

```
use libp2p::StreamProtocol;

// 버전별 프로토콜 식별자
pub const ECHO_V1: StreamProtocol = StreamProtocol::new("/echo/1.0.0");
pub const ECHO_V2: StreamProtocol = StreamProtocol::new("/echo/2.0.0");
pub const ECHO_V3: StreamProtocol = StreamProtocol::new("/echo/3.0.0");

// 버전 호환성 관리
#[derive(Debug, Clone)]
pub struct VersionedEchoProtocol {
supported_versions: Vec<StreamProtocol>,
}

impl VersionedEchoProtocol {
pub fn new() -> Self {
Self {
supported_versions: vec![ECHO_V3, ECHO_V2, ECHO_V1], // 우선순위 순서
}
}
```

```
pub  fn with_versions(versions: Vec<StreamProtocol>) -> Self {
Self {
supported_versions: versions,
}
}
}

impl UpgradeInfo for VersionedEchoProtocol {
type Info = StreamProtocol;
type InfoIter = std::vec::IntoIter<Self::Info>;

fn protocol_info(&self) -> Self::InfoIter {
self.supported_versions.clone().into_iter()
}
}

// 버전별 처리 로직
impl<TSocket> InboundUpgrade<TSocket> for VersionedEchoProtocol
where
TSocket: AsyncRead + AsyncWrite + Unpin + Send + 'static,
{
type Output = (TSocket, StreamProtocol);
```

```
type Error = io::Error;

type Future = futures::future::Ready<Result<Self::Output, Self::Error>>;



fn upgrade_inbound(self, socket: TSocket, info: Self::Info) -> Self::Future {

futures::future::ready(Ok((socket, info)))

}

}
```

4. 테스트 및 디버깅

4.1 단위 테스트

```
#[cfg(test)]

mod tests {

use  super::*;

use libp2p::core::transport::MemoryTransport;

use libp2p::swarm::SwarmEvent;

use tokio::time::{timeout, Duration};



#[tokio::test]

async  fn test_echo_protocol() {

let local_key = identity::Keypair::generate_ed25519();

let  mut swarm = SwarmBuilder::with_existing_identity(local_key)
```

```
.with_tokio()
.with_other_transport(|_| MemoryTransport::default())?
.with_behaviour(|key| EchoBehaviour::new())?
.build();


let listen_addr = "/memory/1234".parse().unwrap();
swarm.listen_on(listen_addr).unwrap();


// 테스트 로직
let timeout_duration = Duration::from_secs(10);
let result = timeout(timeout_duration, async {
loop {
match swarm.select_next_some().await {
SwarmEvent::NewListenAddr { .. } => {
// 리스닝 시작됨
break;
}
_ => {}
}
}
}).await;


assert!(result.is_ok());
```

```
}

#[tokio::test]
async  fn test_message_serialization() {
let request = CustomRequest {
id: 123,
data: "test data".to_string(),
timestamp: 1234567890,
};

let serialized = bincode::serialize(&request).unwrap();
let deserialized: CustomRequest = bincode::deserialize(&serialized).unwrap();

assert_eq!(request.id, deserialized.id);
assert_eq!(request.data, deserialized.data);
assert_eq!(request.timestamp, deserialized.timestamp);
}
}
```

4.2 통합 테스트

```
#[cfg(test)]
mod integration_tests {
```

```rust
use super::*;
use libp2p::swarm::SwarmEvent;
use std::time::Duration;
use tokio::time::timeout;

#[tokio::test]
async fn test_two_node_communication() {
// 첫 번째 노드 설정
let mut swarm1 = create_swarm().await.unwrap();
swarm1.listen_on("/ip4/127.0.0.1/tcp/0".parse().unwrap()).unwrap();

// 두 번째 노드 설정
let mut swarm2 = create_swarm().await.unwrap();

let mut addr1 = None;
let timeout_duration = Duration::from_secs(30);

// 첫 번째 노드의 주소 획득
let result = timeout(timeout_duration, async {
loop {
match swarm1.select_next_some().await {
SwarmEvent::NewListenAddr { address, .. } => {
addr1 = Some(address);
```

```
break;
}
_ => {}
}
}
}).await;


assert!(result.is_ok());
let addr1 = addr1.unwrap();


// 두 번째 노드에서 첫 번째 노드로 연결
swarm2.dial(addr1).unwrap();


// 연결 및 메시지 교환 테스트
let result = timeout(timeout_duration, async {
let  mut connected = false;
loop {
tokio::select! {
event = swarm1.select_next_some() => {
match event {
SwarmEvent::ConnectionEstablished { .. } => {
connected = true;
}
```

```
SwarmEvent::Behaviour(MyBehaviourEvent::Echo(echo_event)) => {
match echo_event {
EchoBehaviourEvent::MessageReceived { .. } => {
return; // 테스트 성공
}
_ => {}
}
}
_ => {}
}
}
event = swarm2.select_next_some() => {
match event {
SwarmEvent::ConnectionEstablished { peer_id, .. } if connected => {
// Echo 메시지 전송
let message = EchoMessage {
data: b"Integration test message".to_vec(),
};
swarm2.behaviour_mut().echo.send_message(peer_id, message).unwrap();
}
_ => {}
}
}
```

```
}
}
}).await;

assert!(result.is_ok());
}
}
```

5. 베스트 프랙티스

5.1 프로토콜 설계 원칙

1. **명확한 프로토콜 식별자**: 버전 정보를 포함한 고유한 식별자 사용
2. **하위 호환성**: 새 버전에서도 이전 버전과의 호환성 유지
3. **오류 처리**: 네트워크 오류 및 프로토콜 오류에 대한 적절한 처리
4. **리소스 관리**: 메모리 및 연결 리소스의 효율적 관리
5. **보안 고려**: 입력 검증 및 DoS 공격 방어

5.2 성능 최적화

```
// 버퍼 크기 최적화
const BUFFER_SIZE: usize = 8192;
const MAX_MESSAGE_SIZE: usize = 1024 * 1024; // 1MB

// 연결 풀링
```

```
pub struct ConnectionPool {
idle_connections: HashMap<PeerId, Vec<NegotiatedSubstream>>,
max_idle_per_peer: usize,
}

impl ConnectionPool {
pub fn new(max_idle_per_peer: usize) -> Self {
Self {
idle_connections: HashMap::new(),
max_idle_per_peer,
}
}

pub fn get_connection(&mut self, peer: &PeerId) -> Option<NegotiatedSubstream>
self.idle_connections
.get_mut(peer)
.and_then(|connections| connections.pop())
}

pub fn return_connection(&mut self, peer: PeerId, connection: NegotiatedSubstr
let connections = self.idle_connections.entry(peer).or_insert_with(Vec::new);
if connections.len() < self.max_idle_per_peer {
connections.push(connection);
```

```
}
}
}
```

5.3 모니터링 및 메트릭

```
use std::sync::atomic::{AtomicU64, Ordering};
use std::sync::Arc;

#[derive(Debug, Clone)]
pub struct ProtocolMetrics {
messages_sent: Arc<AtomicU64>,
messages_received: Arc<AtomicU64>,
errors: Arc<AtomicU64>,
bytes_sent: Arc<AtomicU64>,
bytes_received: Arc<AtomicU64>,
}

impl ProtocolMetrics {
pub fn new() -> Self {
Self {
messages_sent: Arc::new(AtomicU64::new(0)),
messages_received: Arc::new(AtomicU64::new(0)),
```

```
errors: Arc::new(AtomicU64::new(0)),
bytes_sent: Arc::new(AtomicU64::new(0)),
bytes_received: Arc::new(AtomicU64::new(0)),
}
}

pub  fn record_message_sent(&self, bytes: u64) {
self.messages_sent.fetch_add(1, Ordering::Relaxed);
self.bytes_sent.fetch_add(bytes, Ordering::Relaxed);
}

pub  fn record_message_received(&self, bytes: u64) {
self.messages_received.fetch_add(1, Ordering::Relaxed);
self.bytes_received.fetch_add(bytes, Ordering::Relaxed);
}

pub  fn record_error(&self) {
self.errors.fetch_add(1, Ordering::Relaxed);
}

pub  fn get_stats(&self) -> ProtocolStats {
ProtocolStats {
messages_sent: self.messages_sent.load(Ordering::Relaxed),
```

```
messages_received: self.messages_received.load(Ordering::Relaxed),

errors: self.errors.load(Ordering::Relaxed),

bytes_sent: self.bytes_sent.load(Ordering::Relaxed),

bytes_received: self.bytes_received.load(Ordering::Relaxed),

}

}

}



#[derive(Debug)]

pub  struct ProtocolStats {

pub messages_sent: u64,

pub messages_received: u64,

pub errors: u64,

pub bytes_sent: u64,

pub bytes_received: u64,

}
```

이 예제들은 Rust libp2p에서 커스텀 프로토콜을 구현하고 등록하는 완전한 방법을 보여줍니다. multistream-select를 통한 프로토콜 네고시에이션부터 NetworkBehaviour 통합까지 모든 단계를 포함하고 있습니다.

6. 고급 사용 사례

6.1 스트리밍 데이터 프로토콜

```rust
use futures::{SinkExt, StreamExt};

use libp2p::core::upgrade::{read_length_prefixed, write_length_prefixed};


// 스트리밍 프로토콜 구현

#[derive(Debug, Clone)]

pub  struct StreamingProtocol;


const STREAMING_PROTOCOL: StreamProtocol = StreamProtocol::new("/streaming/1.0.0


impl UpgradeInfo for StreamingProtocol {

type Info = StreamProtocol;

type InfoIter = std::iter::Once<Self::Info>;


fn protocol_info(&self) -> Self::InfoIter {

std::iter::once(STREAMING_PROTOCOL)

}

}


// 스트리밍 데이터 타입

#[derive(Debug, Clone, Serialize, Deserialize)]

pub  struct StreamChunk {

pub sequence: u64,
```

```rust
pub data: Vec<u8>,
pub is_final: bool,
}

// 스트리밍 핸들러
pub struct StreamingHandler {
pending_streams: HashMap<u64, Vec<StreamChunk>>,
next_stream_id: u64,
metrics: ProtocolMetrics,
}

impl StreamingHandler {
pub fn new() -> Self {
Self {
pending_streams: HashMap::new(),
next_stream_id: 0,
metrics: ProtocolMetrics::new(),
}
}

pub async fn send_stream<TSocket>(
&mut self,
mut socket: TSocket,
```

```
data: Vec<u8>,
chunk_size: usize,
) -> Result<(), io::Error>
where
TSocket: AsyncWrite + Unpin,
{
let stream_id = self.next_stream_id;
self.next_stream_id += 1;


let chunks: Vec<_> = data
.chunks(chunk_size)
.enumerate()
.map(|(i, chunk)| StreamChunk {
sequence: i as u64,
data: chunk.to_vec(),
is_final: i == (data.len() + chunk_size - 1) / chunk_size - 1,
})
.collect();


for chunk in chunks {
let serialized = bincode::serialize(&chunk)
.map_err(|e| io::Error::new(io::ErrorKind::InvalidData, e))?;
write_length_prefixed(&mut socket, serialized).await?;
```

```
self.metrics.record_message_sent(chunk.data.len() as u64);
}

Ok(())
}

pub  async  fn receive_stream<TSocket>(
&mut  self,
mut socket: TSocket,
) -> Result<Vec<u8>, io::Error>
where
TSocket: AsyncRead + Unpin,
{
let  mut chunks = Vec::new();
loop {
let data = read_length_prefixed(&mut socket, 1024 * 1024).await?; // 1MB max
let chunk: StreamChunk = bincode::deserialize(&data)
.map_err(|e| io::Error::new(io::ErrorKind::InvalidData, e))?;

self.metrics.record_message_received(chunk.data.len() as u64);
chunks.push(chunk.clone());

if chunk.is_final {
```

```
break;
}
}

// 청크를 순서대로 정렬하고 데이터 재구성
chunks.sort_by_key(|c| c.sequence);
let mut result = Vec::new();
for chunk in chunks {
result.extend(chunk.data);
}

Ok(result)
}
}
```

6.2 Pub/Sub 패턴 프로토콜

```
use std::collections::{HashMap, HashSet};

// Topic 기반 Pub/Sub 프로토콜
#[derive(Debug, Clone)]
pub struct PubSubProtocol;
```

```rust
const PUBSUB_PROTOCOL: StreamProtocol = StreamProtocol::new("/pubsub/1.0.0");


#[derive(Debug, Clone, Serialize, Deserialize)]
pub  enum PubSubMessage {
Subscribe { topic: String },
Unsubscribe { topic: String },
Publish { topic: String, data: Vec<u8> },
Message { topic: String, data: Vec<u8>, sender: PeerId },
}


// PubSub Behaviour
pub  struct PubSubBehaviour {
subscriptions: HashMap<String, HashSet<PeerId>>,
peer_subscriptions: HashMap<PeerId, HashSet<String>>,
pending_messages: VecDeque<(PeerId, PubSubMessage)>,
events: VecDeque<PubSubEvent>,
}


#[derive(Debug)]
pub  enum PubSubEvent {
Subscribed { peer: PeerId, topic: String },
Unsubscribed { peer: PeerId, topic: String },
MessageReceived { topic: String, data: Vec<u8>, sender: PeerId },
```

```
MessagePublished { topic: String, subscribers: usize },
}

impl PubSubBehaviour {
pub fn new() -> Self {
Self {
subscriptions: HashMap::new(),
peer_subscriptions: HashMap::new(),
pending_messages: VecDeque::new(),
events: VecDeque::new(),
}
}

pub fn subscribe(&mut self, topic: String) {
// 로컬 구독 처리 및 피어들에게 알림
for peer in self.peer_subscriptions.keys() {
self.pending_messages.push_back((
*peer,
PubSubMessage::Subscribe { topic: topic.clone() }
));
}
}
```

```rust
pub fn publish(&mut self, topic: String, data: Vec<u8>) {
if let Some(subscribers) = self.subscriptions.get(&topic) {
let message = PubSubMessage::Message {
topic: topic.clone(),
data,
sender: self.local_peer_id(), // 실제 구현에서는 local peer id 필요
};

for &peer in subscribers {
self.pending_messages.push_back((peer, message.clone()));
}

self.events.push_back(PubSubEvent::MessagePublished {
topic,
subscribers: subscribers.len(),
});
}
}

fn handle_peer_message(&mut self, peer: PeerId, message: PubSubMessage) {
match message {
PubSubMessage::Subscribe { topic } => {
self.subscriptions.entry(topic.clone()).or_default().insert(peer);
```

```
self.peer_subscriptions.entry(peer).or_default().insert(topic.clone());
self.events.push_back(PubSubEvent::Subscribed { peer, topic });
}
PubSubMessage::Unsubscribe { topic } => {
if  let Some(subscribers) = self.subscriptions.get_mut(&topic) {
subscribers.remove(&peer);
}
if  let Some(topics) = self.peer_subscriptions.get_mut(&peer) {
topics.remove(&topic);
}
self.events.push_back(PubSubEvent::Unsubscribed { peer, topic });
}
PubSubMessage::Message { topic, data, sender } => {
self.events.push_back(PubSubEvent::MessageReceived { topic, data, sender });
}
PubSubMessage::Publish { topic, data } => {
// 다른 피어로부터의 발행 요청 처리
self.publish(topic, data);
}
}
}


fn local_peer_id(&self) -> PeerId {
```

```
// 실제 구현에서는 적절한 방법으로 local peer id 획득
PeerId::random()
}
}
```

6.3 파일 전송 프로토콜

```
use std::path::Path;
use tokio::fs::File;
use tokio::io::{AsyncReadExt, AsyncWriteExt};

// 파일 전송 프로토콜
#[derive(Debug, Clone)]
pub  struct FileTransferProtocol;

const FILE_TRANSFER_PROTOCOL: StreamProtocol = StreamProtocol::new("/file-transf

#[derive(Debug, Clone, Serialize, Deserialize)]
pub  struct FileMetadata {
pub name: String,
pub size: u64,
pub checksum: String, // SHA-256 해시
}
```

```rust
#[derive(Debug, Clone, Serialize, Deserialize)]
pub enum FileTransferMessage {
RequestFile { name: String },
FileMetadata(FileMetadata),
FileChunk { offset: u64, data: Vec<u8> },
FileComplete,
Error { message: String },
}

// 파일 전송 핸들러
pub struct FileTransferHandler {
pending_transfers: HashMap<String, FileTransfer>,
chunk_size: usize,
}

#[derive(Debug)]
struct FileTransfer {
metadata: FileMetadata,
received_bytes: u64,
file_handle: Option<File>,
chunks: HashMap<u64, Vec<u8>>,
}
```

```rust
impl FileTransferHandler {
pub fn new(chunk_size: usize) -> Self {
Self {
pending_transfers: HashMap::new(),
chunk_size,
}
}

pub async fn send_file<TSocket>(
&mut self,
mut socket: TSocket,
file_path: &Path,
) -> Result<(), Box<dyn Error>>
where
TSocket: AsyncRead + AsyncWrite + Unpin,
{
// 파일 메타데이터 읽기
let mut file = File::open(file_path).await?;
let metadata = file.metadata().await?;
let size = metadata.len();
// 체크섬 계산
let mut hasher = sha2::Sha256::new();
```

```
let  mut buffer = vec![0u8; self.chunk_size];

let  mut total_read = 0;

while total_read < size {

let bytes_read = file.read(&mut buffer).await?;

if bytes_read == 0 {

break;

}

hasher.update(&buffer[..bytes_read]);

total_read += bytes_read as u64;

}

let checksum = format!("{:x}", hasher.finalize());

// 파일 메타데이터 전송

let file_metadata = FileMetadata {

name: file_path.file_name().unwrap().to_string_lossy().to_string(),

size,

checksum,

};

let metadata_msg = FileTransferMessage::FileMetadata(file_metadata);

let serialized = bincode::serialize(&metadata_msg)?;

write_length_prefixed(&mut socket, serialized).await?;



// 파일 데이터 전송

file.seek(std::io::SeekFrom::Start(0)).await?;
```

```
let  mut offset = 0;
while offset < size {
let  mut chunk = vec![0u8; self.chunk_size];
let bytes_read = file.read(&mut chunk).await?;
if bytes_read == 0 {
break;
}
chunk.truncate(bytes_read);
let chunk_msg = FileTransferMessage::FileChunk {
offset,
data: chunk,
};
let serialized = bincode::serialize(&chunk_msg)?;
write_length_prefixed(&mut socket, serialized).await?;
offset += bytes_read as u64;
}

// 전송 완료 신호
let complete_msg = FileTransferMessage::FileComplete;
let serialized = bincode::serialize(&complete_msg)?;
write_length_prefixed(&mut socket, serialized).await?;

Ok(())
```

```
}

pub  async  fn receive_file<TSocket>(
&mut  self,
mut socket: TSocket,
output_dir: &Path,
) -> Result<String, Box<dyn Error>>
where
TSocket: AsyncRead + AsyncWrite + Unpin,
{
let  mut file_transfer: Option<FileTransfer> = None;
let  mut output_file: Option<File> = None;

loop {
let data = read_length_prefixed(&mut socket, 1024 * 1024).await?;
let message: FileTransferMessage = bincode::deserialize(&data)?;

match message {
FileTransferMessage::FileMetadata(metadata) => {
let file_path = output_dir.join(&metadata.name);
let file = File::create(&file_path).await?;
file_transfer = Some(FileTransfer {
metadata: metadata.clone(),
```

```
received_bytes: 0,
file_handle: None,
chunks: HashMap::new(),
});
output_file = Some(file);
}
FileTransferMessage::FileChunk { offset, data } => {
if let (Some(ref mut transfer), Some(ref mut file)) =
(&mut file_transfer, &mut output_file) {
file.seek(std::io::SeekFrom::Start(offset)).await?;
file.write_all(&data).await?;
transfer.received_bytes += data.len() as u64;
}
}
FileTransferMessage::FileComplete => {
if let Some(transfer) = file_transfer {
if let Some(mut file) = output_file {
file.flush().await?;
// 체크섬 검증
file.seek(std::io::SeekFrom::Start(0)).await?;
let mut hasher = sha2::Sha256::new();
let mut buffer = vec![0u8; self.chunk_size];
loop {
```

```
let bytes_read = file.read(&mut buffer).await?;

if bytes_read == 0 {

break;

}

hasher.update(&buffer[..bytes_read]);

}

let calculated_checksum = format!("{:x}", hasher.finalize());

if calculated_checksum == transfer.metadata.checksum {

return Ok(transfer.metadata.name);

} else {

return Err("Checksum verification failed".into());

}

}

}

}

FileTransferMessage::Error { message } => {

return Err(message.into());

}

_ => {}

}

}

}
```

```
}
```

7. 실제 애플리케이션 예제

7.1 분산 채팅 애플리케이션

```
// 채팅 애플리케이션 통합 예제
use libp2p::{gossipsub, identify, kad, noise, tcp, yamux, Multiaddr, PeerId, Swa
use tokio::io::{self, AsyncBufReadExt, BufReader};

#[derive(NetworkBehaviour)]
pub struct ChatBehaviour {
gossipsub: gossipsub::Behaviour,
identify: identify::Behaviour,
kademlia: kad::Kademlia<kad::store::MemoryStore>,
echo: EchoBehaviour,
file_transfer: FileTransferBehaviour,
}

impl ChatBehaviour {
pub fn new(local_key: &identity::Keypair) -> Result<Self, Box<dyn Error>> {
let local_peer_id = PeerId::from(local_key.public());
```

```
// Gossipsub 설정
let gossipsub_config = gossipsub::ConfigBuilder::default()
.heartbeat_interval(Duration::from_secs(10))
.validation_mode(gossipsub::ValidationMode::Strict)
.build()
.expect("Valid config");
let  mut gossipsub = gossipsub::Behaviour::new(
gossipsub::MessageAuthenticity::Signed(local_key.clone()),
gossipsub_config,
)?;


// 채팅 토픽 구독
let chat_topic = gossipsub::IdentTopic::new("chat");
gossipsub.subscribe(&chat_topic)?;


// Identify 설정
let identify = identify::Behaviour::new(
identify::Config::new("/chat-app/1.0.0".to_string(), local_key.public())
);


// Kademlia 설정
let store = kad::store::MemoryStore::new(local_peer_id);
let  mut kademlia = kad::Kademlia::new(local_peer_id, store);
```

```
kademlia.set_mode(Some(kad::Mode::Server));


Ok(Self {
gossipsub,
identify,
kademlia,
echo: EchoBehaviour::new(),
file_transfer: FileTransferBehaviour::new(),
})
}


pub fn publish_message(&mut self, message: &str) -> Result<(), Box<dyn Error>>
let chat_topic = gossipsub::IdentTopic::new("chat");
self.gossipsub.publish(chat_topic, message.as_bytes())?;
Ok(())
}
}


// 채팅 애플리케이션 메인 함수
pub async fn run_chat_app() -> Result<(), Box<dyn Error>> {
let local_key = identity::Keypair::generate_ed25519();
let local_peer_id = PeerId::from(local_key.public());
```

```rust
let  mut swarm = SwarmBuilder::with_existing_identity(local_key)
.with_tokio()
.with_tcp(
tcp::Config::default(),
noise::Config::new,
yamux::Config::default,
)?
.with_behaviour(|key| ChatBehaviour::new(key))?
.with_swarm_config(|c| c.with_idle_connection_timeout(Duration::from_secs(60)))
.build();


// 리스닝 설정
swarm.listen_on("/ip4/0.0.0.0/tcp/0".parse()?)?;


println!("Chat application started. Local peer ID: {}", local_peer_id);
println!("Commands:");
println!(" /connect <multiaddr> - Connect to a peer");
println!(" /send <message> - Send a chat message");
println!(" /file <path> - Send a file");
println!(" /quit - Exit the application");


// 사용자 입력 처리를 위한 stdin reader
let stdin = io::stdin();
```

```rust
let  mut stdin_reader = BufReader::new(stdin).lines();



loop {

tokio::select! {

// 네트워크 이벤트 처리

event = swarm.select_next_some() => {

match event {

SwarmEvent::NewListenAddr { address, .. } => {

println!("Listening on {}", address);

}

SwarmEvent::ConnectionEstablished { peer_id, .. } => {

println!("Connected to {}", peer_id);

}

SwarmEvent::Behaviour(ChatBehaviourEvent::Gossipsub(gossipsub::Event::Message {

propagation_source: peer_id,

message_id: _,

message,

})) => {

let msg = String::from_utf8_lossy(&message.data);

println!("[{}]: {}", peer_id, msg);

}

SwarmEvent::Behaviour(ChatBehaviourEvent::Echo(echo_event)) => {

match echo_event {
```

```
EchoBehaviourEvent::MessageReceived { peer, message } => {
println!("Echo from {}: {:?}", peer, String::from_utf8_lossy(&message.data));
}
_ => {}
}
}
SwarmEvent::Behaviour(ChatBehaviourEvent::FileTransfer(file_event)) => {
match file_event {
FileTransferEvent::FileReceived { peer, filename } => {
println!("File received from {}: {}", peer, filename);
}
FileTransferEvent::FileSent { peer, filename } => {
println!("File sent to {}: {}", peer, filename);
}
_ => {}
}
}
_ => {}
}
}
// 사용자 입력 처리
line = stdin_reader.next_line() => {
if  let Ok(Some(line)) = line {
```

```
let parts: Vec<&str> = line.trim().split_whitespace().collect();

match parts.get(0) {

Some(&"/connect") => {

if  let Some(addr_str) = parts.get(1) {

match addr_str.parse::<Multiaddr>() {

Ok(addr) => {

if  let Err(e) = swarm.dial(addr) {

println!("Failed to dial: {}", e);

}

}

Err(e) => println!("Invalid address: {}", e),

}

} else {

println!("Usage: /connect <multiaddr>");

}

}

Some(&"/send") => {

if parts.len() > 1 {

let message = parts[1..].join(" ");

if  let Err(e) = swarm.behaviour_mut().publish_message(&message) {

println!("Failed to send message: {}", e);

}

} else {
```

```
println!("Usage: /send <message>");
}
}
Some(&"/file") => {
if  let Some(file_path) = parts.get(1) {
// 파일 전송 로직 (실제 구현에서는 대상 피어 선택 필요)
println!("File transfer not implemented in this example");
} else {
println!("Usage: /file <path>");
}
}
Some(&"/quit") => {
println!("Goodbye!");
break;
}
_ => {
// 일반 메시지로 처리
if !line.trim().is_empty() {
if  let Err(e) = swarm.behaviour_mut().publish_message(&line) {
println!("Failed to send message: {}", e);
}
}
}
```

```
}
}
}
}
}

Ok(())
}

// 애플리케이션 진입점
#[tokio::main]
async fn main() -> Result<(), Box<dyn Error>> {
env_logger::init();
run_chat_app().await
}
```

8. 결론

이 문서에서는 Rust libp2p에서 커스텀 프로토콜을 구현하고 등록하는 다양한 방법을 살펴보았습니다:

1. **기본 프로토콜 구현**: Echo 프로토콜을 통한 기본적인 구현 패턴
2. **고급 프로토콜 패턴**: Request-Response, 스트리밍, Pub/Sub, 파일 전송
3. **NetworkBehaviour 통합**: 여러 프로토콜을 조합한 복합 애플리케이션

4. **실제 사용 사례**: 분산 채팅 애플리케이션 예제

핵심 포인트

- **multistream-select**를 통한 프로토콜 네고시에이션
- **UpgradeInfo**, **InboundUpgrade**, **OutboundUpgrade** 트레이트 구현
- **ConnectionHandler**를 통한 연결별 프로토콜 처리
- **NetworkBehaviour**를 통한 전체 네트워크 동작 조율
- 버전 관리, 오류 처리, 성능 최적화 고려사항

이러한 패턴들을 활용하면 libp2p 생태계에 완전히 통합된 커스텀 프로토콜을 구현할 수 있습니다.